LocalShield

Privacy Toolkit Guide

Your data. Your machine. Your rules.

Version 1.0 | 10 Python Privacy Scripts | Zero Cloud Dependencies

Contents

#	Section
1	Why LocalShield Exists
2	Who This Is For
3	Installation
4	Tool 1 — Image Metadata Stripper
5	Tool 2 — File Encryption
6	Tool 3 — PII Scanner
7	Tool 4 — CSV Anonymizer
8	Tool 5 — PDF Redaction
9	Tool 6 — Password Generator
10	Tool 7 — Secure File Deletion
11	Tool 8 — File Integrity Verification
12	Tool 9 — Log File Sanitizer
13	Tool 10 — Office Metadata Stripper
14	Privacy Best Practices Beyond the Scripts
15	Troubleshooting & FAQ

1. Why LocalShield Exists

Every file you create carries invisible metadata. A photo taken on your phone embeds your GPS coordinates, device model, and exact timestamp. A Word document records your name, your organization, and every revision you've ever made. A server log file contains IP addresses, authentication tokens, and connection strings that map your infrastructure.

Most of this data is invisible to you — but it's trivially easy for anyone else to extract.

The privacy tools that exist today often require you to upload your sensitive files to a remote server but innately defeats the purpose.

LocalShield takes a different approach.

- **100% local execution.** Every script runs on your machine.
- **Readable source code.** Every tool is a plain Python script you can open in any text editor and read line by line.
- **10 focused tools.** Each script does one thing well.

2. Who This Is For

LocalShield was designed for professionals and individuals who need practical, reliable and easy to use security tools.

3. Installation

LocalShield requires **Python 3.8 or later**.

Follow the steps below to get up and running:

Step 1: Check Your Python Version

Open your terminal (or Command Prompt on Windows) and run:

python --version

You should see output like Python 3.10.4 or higher. If the command is not found or reports a version below 3.8, install Python using instructions for your platform.

(Note: On some systems, the command is python3 instead of python. Use whichever responds with a 3.x version).

Step 2: Install Dependencies

Copy the LocalShield directory to a folder and install the required packages:

```
cd localshield
pip install -r requirements.txt
```

This installs everything you need. The full dependency list is below.

Dependencies Reference

Package	Purpose	Used By
Pillow	Image processing (EXIF data extraction and removal)	exif_strip.py
cryptography	AES-256-GCM encryption and PBKDF2 key derivation	file_encrypt.py
pandas	CSV reading, analysis, and transformation	csv_anonymize.py
Faker	Realistic fake data generation (names, emails, SSNs)	csv_anonymize.py
PyMuPDF	PDF parsing and text redaction	pdf_redact.py
python-docx	Word document metadata reading and removal	meta_wipe.py
openpyxl	Excel file metadata reading and removal	meta_wipe.py

Note: MVP Testing on Windows only.

Tool 1 — Image Metadata Stripper

exif_strip.py

Why This Matters

Every photo taken on a smartphone embeds EXIF metadata: GPS coordinates, camera model, lens settings, timestamps, and sometimes even your name. When you share a photo online, you may be sharing your exact location — down to the street address —

without realizing it. EXIF data has been used to stalk individuals, identify confidential locations, and correlate identities across platforms.

exif_strip.py removes all EXIF metadata from image files while leaving the image itself completely untouched.

Basic Usage

python exif_strip.py photo.jpg

This strips all metadata from photo.jpg and saves the cleaned version in place.

Key Flags & Options

Flag	Description	Example
-r	Recursively process all images in a directory	python exif_strip.py -r ./photos/
--preview	Show what metadata would be removed without modifying the file	python exif_strip.py --preview photo.jpg
-o	Specify output path (preserves the original)	python exif_strip.py photo.jpg -o clean.jpg
-v	Verbose output — lists every metadata field removed	python exif_strip.py -v photo.jpg

Pro Tip: Run --preview first to see exactly what's embedded in your photos. You may be surprised by how much data your phone records. Batch-process an entire folder before uploading to social media or sending to clients with -r.

Tool 2 — File Encryption

file_encrypt.py

Why This Matters

If your laptop is stolen, lost, or accessed by an unauthorized person, any unencrypted file is immediately readable. Tax returns, contracts, medical records, client data — all exposed. Full-disk encryption helps, but it only protects the drive when the machine is

powered off. Individual file encryption gives you a second layer: even if someone gains access to your filesystem, encrypted files remain unreadable without your password.

file_encrypt.py uses **AES-256-GCM** encryption — the same standard used by governments and financial institutions — with **PBKDF2 key derivation** at 600,000 iterations to protect against brute-force attacks.

Basic Usage

Encrypt a file:

```
python file_encrypt.py encrypt sensitive_report.pdf
```

You'll be prompted to enter a password. The encrypted file is saved as sensitive_report.pdf.enc.

Decrypt a file:

```
python file_encrypt.py decrypt sensitive_report.pdf.enc
```

Key Flags & Options

Flag	Description	Example
encrypt	Encrypt the specified file	python file_encrypt.py encrypt data.csv
decrypt	Decrypt a previously encrypted file	python file_encrypt.py decrypt data.csv.enc
-o	Specify a custom output path	python file_encrypt.py encrypt data.csv -o secure_data.enc

Warning

Do not lose your password. AES-256-GCM encryption is irreversible without the correct password. There is no recovery mechanism, no backdoor, and no reset option. Store your password in a password manager.

Pro Tip: Encrypt sensitive files before uploading them to cloud storage (Google Drive, Dropbox, OneDrive). Even if the cloud provider is breached, your files remain protected. Combine with hash_verify.py to confirm file integrity after transfer.

Tool 3 — PII Scanner

data_scan.py

Why This Matters

Personally Identifiable Information (PII) hides in places you don't expect: a CSV export with a Social Security Number column you forgot about, an email address buried in a log file, a credit card number pasted into a text document during testing. Sharing files that contain PII — even accidentally — can violate privacy regulations, breach client trust, and create legal liability.

data_scan.py scans text files and reports every instance of PII it finds, including:

- **Social Security Numbers** (XXX-XX-XXXX pattern)
- **Credit card numbers** (Luhn algorithm validated — not just pattern matching)
- **Email addresses**
- **Phone numbers** (US formats)
- **IP addresses** (IPv4)

Basic Usage

python data_scan.py document.txt

Key Flags & Options

Flag	Description	Example
-r	Recursively scan all files in a directory	python data_scan.py -r ./project_data/
-o	Save results to a file instead of printing to screen	python data_scan.py -r ./data/ -o scan_report.txt
-v	Verbose — shows line numbers and surrounding context	python data_scan.py -v document.txt

Pro Tip

Make PII scanning part of your workflow: scan every file before sending it to a third party. Run data_scan.py -r on your project directory before committing to version control. Pipe the output to JSON for integration with your own scripts.

Tool 4 — CSV Anonymizer

csv_anonymize.py

Why This Matters

Spreadsheets are the backbone of business data — and they're full of real names, real email addresses, real phone numbers, and real Social Security Numbers. When you need to share a dataset for analysis, testing, or collaboration, you need the *structure* of the data without the *identity* of the people in it.

Simply deleting columns destroys the dataset's usefulness. csv_anonymize.py takes a smarter approach: it replaces real PII with **realistic fake data** generated by the Faker library. Names become different names. Emails become different emails. The data structure, column relationships, and statistical properties stay intact.

Basic Usage

python csv_anonymize.py employees.csv -o employees_anonymized.csv

Key Flags & Options

Flag	Description	Example
--preview	Show a before/after comparison without modifying files	python csv_anonymize.py --preview employees.csv
--columns	Manually specify which columns to anonymize	python csv_anonymize.py --columns name,email,ssn data.csv
-o	Specify output file path	python csv_anonymize.py data.csv -o clean_data.csv

Referential Integrity

The anonymizer maintains referential integrity across the file. If "Jane Smith" appears in 15 rows, every instance is replaced with the *same* fake name. This preserves relationships and patterns in the data while fully de-identifying individuals.

Pro Tip

Always use --preview first to verify the auto-detection is identifying the right columns. If a column is missed, add it manually with --columns.

Tool 5 — PDF Redaction

pdf_redact.py

Why This Matters

pdf_redact.py performs true, permanent redaction. It identifies the specified text, removes it from the PDF's content stream, and renders a black box in its place. The original text is gone — not hidden, not layered, gone.

Basic Usage

Redact by exact text match:

```
python pdf_redact.py document.pdf --text "John Smith"
```

Redact by regex pattern (e.g., all SSNs):

```
python pdf_redact.py document.pdf --pattern "\d{3}-\d{2}-\d{4}"
```

Key Flags & Options

Flag	Description	Example
--text	Redact exact text matches	--text "Confidential"
--pattern	Redact text matching a regex pattern	--pattern "\b\d{3}-\d{2}-\d{4}\b"
-o	Specify output file (default overwrites original)	-o redacted_document.pdf

Warning — Irreversible

PDF redaction is **permanent and irreversible**. Always work on a copy of your document or use the -o flag to write to a new file. There is no undo.

Pro Tip

Combine --text and --pattern in the same command to redact both known names and structured patterns (like SSNs or account numbers) in a single pass. Essential for legal document preparation and FOIA responses.

Tool 6 — Password Generator

password_forge.py

Why This Matters

Weak, reused passwords are the single most common security vulnerability for individuals and small businesses. Yet generating truly random, high-entropy passwords is something humans are fundamentally bad at. We gravitate toward patterns, dictionary words, and predictable substitutions.

password_forge.py generates cryptographically secure passwords in three modes, displays the entropy (a measure of password strength), and never sends anything over the network.

Basic Usage

Random password (default — 20 characters):

python password_forge.py random

Passphrase (word-based, easy to type):

python password_forge.py passphrase

PIN (numeric only):

python password_forge.py pin

Key Flags & Options

Flag	Description	Example
random	Generate a random password with configurable character set	python password_forge.py -m random -l 32
passphrase	Generate a word-based passphrase (e.g., correct-horse-battery-staple)	python password_forge.py -m passphrase -w 6
pin	Generate a numeric PIN	python password_forge.py -m pin -l 8
--exclude-ambiguous	Remove visually ambiguous characters (0/O, 1/l/I)	python password_forge.py random --exclude-ambiguous
-l	Set length (characters for random/pin, words for passphrase)	python password_forge.py random -l 24

Entropy

Every generated password displays its entropy in bits. As a rule of thumb: 40 bits is weak, 60 bits is adequate, 80+ bits is strong, and 128+ bits is excellent. The default settings produce passwords well above the strong threshold.

Pro Tip

Use passphrase mode for passwords you need to type manually (like a master password). Use random mode for passwords stored in a password manager. Use --exclude-ambiguous when the password will be read aloud or transcribed on paper.

Tool 7 — Secure File Deletion

file_shred.py

Why This Matters

When you "delete" a file, your operating system only removes the pointer to it. The actual data remains on disk until it's overwritten by new data — which could take weeks, months,

or never. Anyone with a data recovery tool can retrieve "deleted" files from a hard drive, USB stick, or discarded laptop.

file_shred.py performs **secure deletion**: it overwrites the file's contents with random data across multiple passes, then deletes it. This makes recovery effectively impossible on traditional hard drives.

Basic Usage

```
python file_shred.py sensitive_document.pdf
```

You'll be asked to confirm before deletion proceeds.

Key Flags & Options

Flag	Description	Example
--passes	Number of overwrite passes (default: 3)	python file_shred.py --passes 7 file.pdf
--dry-run	Preview what would be deleted without actually deleting	python file_shred.py --dry-run ./old_records/
--force	Skip the confirmation prompt	python file_shred.py --force temp_data.csv
-r	Recursively shred all files in a directory	python file_shred.py -r ./temp_project/

Warning — Permanent Destruction

Securely shredded files **cannot be recovered**. Always use --dry-run first to verify you're targeting the right files. There is no recycle bin, no undo, and no recovery.

Pro Tip

Use this tool when decommissioning a project: shred temporary data files, exported credentials, and intermediate processing files. For SSDs (solid-state drives), note that wear-leveling can complicate secure deletion at the hardware level — for SSDs, combine shredding with full-disk encryption for best results.

Tool 8 — File Integrity Verification

hash_verify.py

Why This Matters

How do you know a file hasn't been tampered with? When you download software, receive a document via email, or transfer files between systems, you have no guarantee the file that arrived is identical to the file that was sent. A single bit change could mean corruption — or it could mean malicious modification.

hash_verify.py generates and verifies cryptographic hashes (digital fingerprints) for files. If even one byte changes, the hash changes completely, making tampering immediately detectable.

Basic Usage

Generate a hash:

```
python hash_verify.py generate report.pdf
```

Verify a file against a known hash:

```
python hash_verify.py verify report.pdf abc123def456...
```

Compare two files:

```
python hash_verify.py compare original.pdf received.pdf
```

Key Flags & Options

Flag	Description	Example
generate	Compute and display the hash of a file	python hash_verify.py generate file.zip
verify	Check a file against a known hash value	python hash_verify.py verify file.zip <hash>
compare	Compare two files to see if they're identical	python hash_verify.py compare a.pdf b.pdf
--algorithm	Choose hash algorithm (default: sha256)	python hash_verify.py generate file.zip --algorithm sha512

Pro Tip

After encrypting a file with file_encrypt.py, generate its hash and send the hash through a separate channel (e.g., text message). The recipient can verify the encrypted file wasn't corrupted or tampered with during transfer before decrypting. SHA-256 is the recommended default for most use cases.

Tool 9 — Log File Sanitizer

log_cleaner.py

Why This Matters

Application log files are a goldmine of sensitive data. IP addresses reveal user locations. Email addresses identify individuals. Tokens and API keys grant unauthorized access if exposed. AWS keys, bearer tokens, and database connection strings are routinely logged during development and debugging — and routinely forgotten when those logs are shared with contractors, posted in bug reports, or committed to repositories.

log_cleaner.py automatically detects and replaces sensitive data in log files with consistent, readable placeholders. IP addresses become [IP_REDACTED_1], emails become [EMAIL_REDACTED_1], and so on — with consistent numbering so you can still trace patterns without exposing real data.

Basic Usage

```
python log_cleaner.py application.log -o sanitized.log
```

Detected Data Types

Data Type	Placeholder Format
IP Addresses (IPv4)	[IP_REDACTED_N]
Email Addresses	[EMAIL_REDACTED_N]
AWS Access Keys	[AWS_KEY_REDACTED]
Bearer Tokens	[BEARER_REDACTED]
Connection Strings	[CONN_STRING_REDACTED]

Key Flags & Options

Flag	Description	Example
--preview	Show what would be sanitized without modifying the file	python log_cleaner.py --preview server.log
-o	Specify output file	python log_cleaner.py server.log -o clean.log

Pro Tip

Always sanitize log files before attaching them to support tickets, bug reports, or sharing with external developers. A single exposed AWS key in a log file can lead to thousands of dollars in unauthorized cloud charges within hours.

Tool 10 — Office Metadata Stripper

meta_wipe.py

Why This Matters

Microsoft Office documents — Word files (.docx) and Excel files (.xlsx) — store a surprising amount of metadata beyond the visible content. Author names, organization names, editing time, revision counts, last-saved-by usernames, comments, tracked changes, and template paths are all embedded in the file. This metadata can reveal internal project names, employee identities, organizational structure, and editing history to anyone who receives the document.

meta_wipe.py strips all metadata from Office documents, leaving only the visible content.

Basic Usage

python meta_wipe.py contract_final.docx

Key Flags & Options

Flag	Description	Example

--preview	Display all metadata that would be removed, without modifying the file	python meta_wipe.py --preview quarterly_report.xlsx
-o	Save the cleaned file to a new path	python meta_wipe.py report.docx -o report_clean.docx
-r	Recursively process all Office files in a directory	python meta_wipe.py -r ./deliverables/

Metadata Removed

Metadata Field	Why It's a Risk
Author / Last Modified By	Reveals employee names and internal usernames
Organization	Exposes company name when using templates
Revision Count & Edit Time	Shows how much effort went into the document
Comments & Tracked Changes	May contain internal discussions, edits, or sensitive notes
Template Path	Can reveal internal file server paths and directory structure

Pro Tip

Run --preview on a document you've already shared externally. The amount of hidden metadata will likely surprise you. Make metadata stripping a standard step before sending any document outside your organization.

14. Privacy Best Practices Beyond the Scripts

LocalShield gives you powerful tools — but tools are most effective when used as part of consistent habits. Here are six practices to integrate into your daily workflow:

#	Practice	How LocalShield Helps
1	**Strip metadata before sharing.** Every file you send externally should have its metadata removed first — photos, documents, spreadsheets.	exif_strip.py, meta_wipe.py

2	**Encrypt sensitive files at rest.** Tax documents, contracts, medical records, and client data should be encrypted on disk — not just behind a login screen.	file_encrypt.py
3	**Scan before sending.** Before emailing a file or uploading it to a shared drive, scan it for PII. One missed SSN can create a compliance incident.	data_scan.py
4	**Use unique, strong passwords.** Every account gets its own password. Use a password manager and generate passwords with high entropy.	password_forge.py
5	**Verify file integrity after transfers.** When you receive an important file, verify its hash matches what the sender published. Corruption and tampering are both real risks.	hash_verify.py
6	**Sanitize logs before sharing with developers.** Log files routinely contain secrets. Clean them before attaching to tickets, posting in forums, or sending to contractors.	log_cleaner.py

15. Troubleshooting & FAQ

Common Issues

Error	Cause	Solution
ModuleNotFoundError: No module named 'PIL'	Required Python packages are not installed	Run pip install -r requirements.txt from the LocalShield directory
Permission denied	The script lacks read/write access to the target file	Check file permissions. On Linux/Mac, try chmod 644 filename. On Windows, right-click → Properties → Security
No such file or directory: 'exif_strip.py'	Your terminal is not in the scripts directory	Navigate to the scripts folder: cd localshield/scripts/
Python was not found or version below 3.8	Python is not installed or an	Install Python 3.8+ from python.org (see Section 3 for platform-specific instructions)

	outdated version is being used	
SyntaxError when running a script	You may be running Python 2 instead of Python 3	Use python3 instead of python, or update your PATH

— LocalShield